

The Bigram Language Model

Building a character-level name generator by counting — and the exact arithmetic behind every number, with this dataset's real values. After Andrej Karpathy's makemore.

A **language model** assigns probabilities to sequences and can generate new ones. We will build the simplest possible one — a **bigram** model — on a dataset of **32,033 names**, and trace every number it produces. The bigram model makes one strong assumption: the next character depends **only on the current one**. That is all it knows, and it is enough to start generating name-like strings.

The whole model in one line

Count how often each character follows each other, then turn those counts into probabilities by dividing. That is the entire bigram model. Everything below is unpacking that sentence carefully.

1. The vocabulary and the boundary token

Characters are not numbers, so we map each of the 26 letters to an integer 1..26, and add one special token `.` at index `0`. This `.` marks both the **start** and the **end** of a name. The vocabulary size is therefore **27**. The boundary token lets a single mechanism learn which letters tend to **begin** names and which tend to **end** them.

Every name is wrapped with the boundary token before counting. The name **emma** becomes the sequence:

`.` → `e` → `m` → `m` → `a` → `.`

which contains the five adjacent pairs (bigrams): `.e`, `em`, `mm`, `ma`, `a.`

2. Counting bigrams into the matrix N

We make a 27×27 table **N**, where **N[i, j]** is the number of times character **j** follows character **i** across all names. Rows are the *current* character; columns are the *next* one.

```
N = torch.zeros((27, 27), dtype=torch.int32)
for w in words:
    chs = ['.'] + list(w) + ['.']          # wrap with boundary tokens
    for ch1, ch2 in zip(chs, chs[1:]):    # slide over adjacent pairs
        N[stoi[ch1], stoi[ch2]] += 1      # tally the transition i -> j
```

On this dataset that records **228,146** bigrams in total (the sum of every cell in **N**).

3. The confusion everyone hits: which denominator?

This is the single most common point of confusion, so we settle it with the real numbers. Two cells look similar but mean opposite things:

- `.a` = **N[0, a] = 4,410** — transition *start* → *a*, i.e. the number of names whose **first** letter is 'a'.

- $a. = N[a, 0] = 6,640$ — transition $a \rightarrow \text{end}$, i.e. the number of names whose **last** letter is 'a'.

Read the arrow direction, not just the letters

$.a$ and $a.$ are **not** the same. The dot on the **left** means 'a' is the first letter; the dot on the **right** means it is the last. On this data 'a' starts 4,410 names and ends 6,640 of them — it is a far more common ending than beginning.

Now, the probability that a name **starts** with 'a'. We only care about row 0 (the start token), and we normalize by that **row's** sum — **not** by the 228,146 total bigrams:

$$P(\text{first} = a) = \frac{N[0, a]}{\sum_k N[0, k]} = \frac{4410}{32033} = 0.1377$$

Why is the denominator **32,033**? Because row 0 only records *start* $\rightarrow$ *first-letter* transitions, and there is exactly **one of those per name**. So the sum of row 0 equals the number of names: `N[0].sum() == 32033`. Each row has its **own** denominator (its own row sum); you never divide a single row by the grand total.

So, 6640 / 228146 is wrong, and 6640 / 32033 is also the wrong question

6,640 is 'a'-as-**last**-letter, which lives in column 0, not row 0. The first-letter probability uses 4,410 (row 0). And the denominator is the row-0 sum 32,033, not the 228,146 total. Mixing these up is what produces a 'wrong' 0.2073; the correct value is **0.1377**.

The six most likely first letters on this data, by count, are:

a: 4410 k: 2963 m: 2538 j: 2422 s: 2055 d: 1690

4. From counts to probabilities (the whole matrix)

The same row-normalization defines the model for **every** current character. Dividing each row by its own sum gives the conditional distribution of the next character:

$$P(j | i) = \frac{N[i, j]}{\sum_k N[i, k]}$$

These empirical frequencies are the **maximum-likelihood estimate** of the bigram model — the probabilities that make the observed names as likely as possible are simply the proportions we observed.

5. A subtle but fatal bug: keepdims and broadcasting

Normalizing all rows at once is one line, but the shape matters enormously:

```
P = N.float()
P /= P.sum(1, keepdims=True)    # divide each row by its sum
```

- `P.sum(1, keepdims=True)` has shape **(27, 1)**. Dividing (27, 27) by (27, 1) stretches that single column across all 27 columns — normalizing each **row**, which is what we want.
- `P.sum(1)` (without `keepdims`) has shape **(27,)**. Dividing (27, 27) by (27,) broadcasts down the **columns** instead — silently normalizing the wrong axis and destroying the model.

Always check the shapes

PyTorch will not error on the broken version — it broadcasts happily and hands you a wrong model. A two-line sanity check (`print(P.sum(1, keepdims=True).shape)`) is the cheapest bug insurance you will ever write.

6. Generating names by sampling

To generate, start at the boundary token (index 0) and repeatedly **sample** the next character from the current row's distribution, then move to that character's row, until we sample `.` again. `torch.multinomial` draws an index in proportion to the probabilities — a weighted die roll.

```
g = torch.Generator().manual_seed(2147483647)
for _ in range(5):
    out, ix = [], 0
    while True:
        p = P[ix]
        ix = torch.multinomial(p, 1, generator=g).item()
        if ix == 0: break
        out.append(itos[ix])
    print(''.join(out))
```

A worked toy example

Suppose a tiny alphabet `{., a, b}` with smoothed, row-normalized probabilities: from the start row, $P(. \rightarrow a) = 4/7$, $P(. \rightarrow b) = 2/7$, $P(. \rightarrow .) = 1/7$. We sample the start row and get **a** (the most likely). From row **a** we sample again and get **b**. From row **b** we sample and get `.` (end). The collected letters spell the generated name “**ab**”. A name is just a random walk through the probability table, starting and ending at the boundary token.

7. Scoring the model in one number: the loss

To compare models we need a single score for “how well does this explain the data.” The natural choice is the **likelihood**: the probability the model assigns to the whole training set, which is the product over every bigram:

$$\text{likelihood} = \prod_{(i,j)} P(j \mid i)$$

That product of hundreds of thousands of small numbers underflows to zero on a computer, so we take its **log** (turning the product into a sum), and because we minimize losses by convention we **negate** and **average** it. The result is the **average negative log-likelihood (NLL)**:

$$\text{loss} = -\frac{1}{n} \sum_{(i,j)} \log P(j | i)$$

- The log of a probability is ≤ 0 , so the NLL is ≥ 0 . **Lower is better.**
- A **perfect** model (probability 1 on every true next character) scores **0**.
- A correct-and-confident prediction (p near 1) contributes near 0; a confident-and-wrong one (p near 0) contributes a huge number. This NLL is exactly the **cross-entropy** between the data and the model.

On this dataset the bigram model scores:

$$\text{loss} \approx 2.45 \qquad \log 27 \approx 3.30 \text{ (uniform)}$$

So the model is well below the random-guessing baseline — it has genuinely learned structure. A model worse than 3.30 would be worse than random (confidently wrong).

8. Perplexity: an easier-to-read version of the loss

Perplexity is just the exponential of the loss, and reads as “the effective number of choices the model is confused among”:

$$\text{perplexity} = e^{\text{loss}} = e^{2.45} \approx 11.6$$

A uniform model over 27 symbols has perplexity 27 (it is “confused among 27”). Our bigram model cut that to about 11.6 — more than halving the uncertainty.

9. Smoothing: why we add +1

If some bigram never appears, its count is 0 and its probability is 0. The trouble is not the division — $0 / 32033 = 0$, which is fine — it is the **log** in the loss: $\log(0) = -\infty$. A single unseen pair would make the entire loss infinite. **Add- α (Laplace) smoothing** fixes this by adding a small constant to every count before normalizing:

$$P(j | i) = \frac{N[i, j] + \alpha}{\sum_k (N[i, k] + \alpha)}$$

- $\alpha = 0$: the raw, brittle maximum-likelihood estimate — one unseen pair and the loss explodes.
- $\alpha = 1$: the common ‘+1’ smoothing — every transition gets at least a tiny probability, so nothing is impossible and the loss stays finite.
- **Large α** : pushes every row toward **uniform** — more smoothing, a less confident model.

The misconception to drop

It is not that 'dividing by zero gives infinity' here. The zero is in the **numerator**: the count is zero, the probability is zero, and it is **log(0)** that is $-\infty$. Smoothing keeps every probability strictly positive so the logarithm is always finite.

10. The same model as a one-layer neural network

Here is the pivot that powers the rest of the series: instead of **counting**, we can **learn** the bigram probabilities by gradient descent — and arrive at the identical model. To feed a character into a network we **one-hot encode** it: index **i** becomes a length-27 vector that is 1 at position **i** and 0 everywhere else.

The 'network' is a single 27×27 weight matrix **W** with no bias and no non-linearity. The forward pass is one matrix multiply producing **logits** (raw scores):

$$\text{logits} = x_{\text{onehot}} W$$

What the zeros are doing

A one-hot vector has a single 1 and twenty-six 0s. The zeros are not 'converted' to anything and they do not erase the character — they are placeholders that make the vector length 27. Because the vector is one-hot, the product **x·W** **simply selects row i of W**. So row **i** of **W** is the neural analogue of row **i** of the count matrix: the scores for what follows character **i**.

Turning logits into probabilities is **softmax** — exponentiate, then normalize:

$$p_j = \frac{e^{\text{logit}_j}}{\sum_k e^{\text{logit}_k}}$$

If we read the logits as **log-counts**, then $\exp(\text{logits})$ are the counts and dividing by their sum is exactly the row-normalization from section 4. So this one-layer softmax network **is** the bigram model; training only has to discover $W \approx \log(N)$.

11. Training W by gradient descent

Every step of the forward pass is differentiable, so autograd gives us the gradient of the loss with respect to **W**. The loss is the same average NLL, now selecting the probability the network assigned to each **true** next character. The loop is the familiar three steps:

```
logits = xenc @ W                      # log-counts (xenc is one-hot)
counts = logits.exp()
probs = counts / counts.sum(1, keepdims=True)      # softmax
loss = -probs[torch.arange(num), ys].log().mean() + 0.01*(W**2).mean()
W.grad = None; loss.backward()              # backward fills W.grad
W.data += -50 * W.grad                      # step downhill
```

Starting from random **W** the loss begins near the uniform baseline (~3.3) and descends to **~2.45** — the very same number the counting model achieved, because they are the same model fit two ways.

12. Regularization is smoothing, rediscovered

The extra term $\lambda \cdot \text{mean}(W^2)$ (here $\lambda = 0.01$) is **L2 regularization**: a penalty on large weights that pulls the optimizer toward $W \rightarrow 0$. And $W = 0$ makes logits = 0, so counts = $e^0 = 1$, so the distribution is **uniform**. That is precisely what add-one smoothing did to the count model. Same knob, two formalisms:

- Turn λ (or α) **up** $\rightarrow$ the model is smoothed toward uniform, samples get more random.
- Turn it to **0** $\rightarrow$ the model fits the data as hard as it can.

13. Going further: trigrams and quadgrams

A bigram uses one character of context. To use more, you change **only** the input representation; the softmax, loss, and gradient descent are untouched.

- **Trigram**: predict from the previous **two** characters. Encode the pair (i_{t-2}, i_{t-1}) as a single index $i_{t-2} \cdot 27 + i_{t-1}$, giving $27^2 = 729$ possible contexts; W becomes 729×27 .
- **Quadgram**: predict from the previous **three**. Index $i_{t-3} \cdot 729 + i_{t-2} \cdot 27 + i_{t-1}$, giving $27^3 = 19,683$ contexts; W becomes $19,683 \times 27$.

Why this approach runs out of road

Each extra character multiplies the number of rows by 27. The table explodes, and most rows are seen only a handful of times (or never) — the counts become unreliable. That sparsity is exactly the problem the **MLP** (Part 2) solves: instead of a row per context, it learns a small dense **embedding** per character and a neural network on top, so similar contexts share statistical strength. Same softmax and cross-entropy loss — a smarter front end.

Companion: the runnable notebook [Bigram_explained.ipynb](#) and the interactive visualizer [Bigram, Animated](#) (th3samaritan.github.io/Al-Research/bigram).